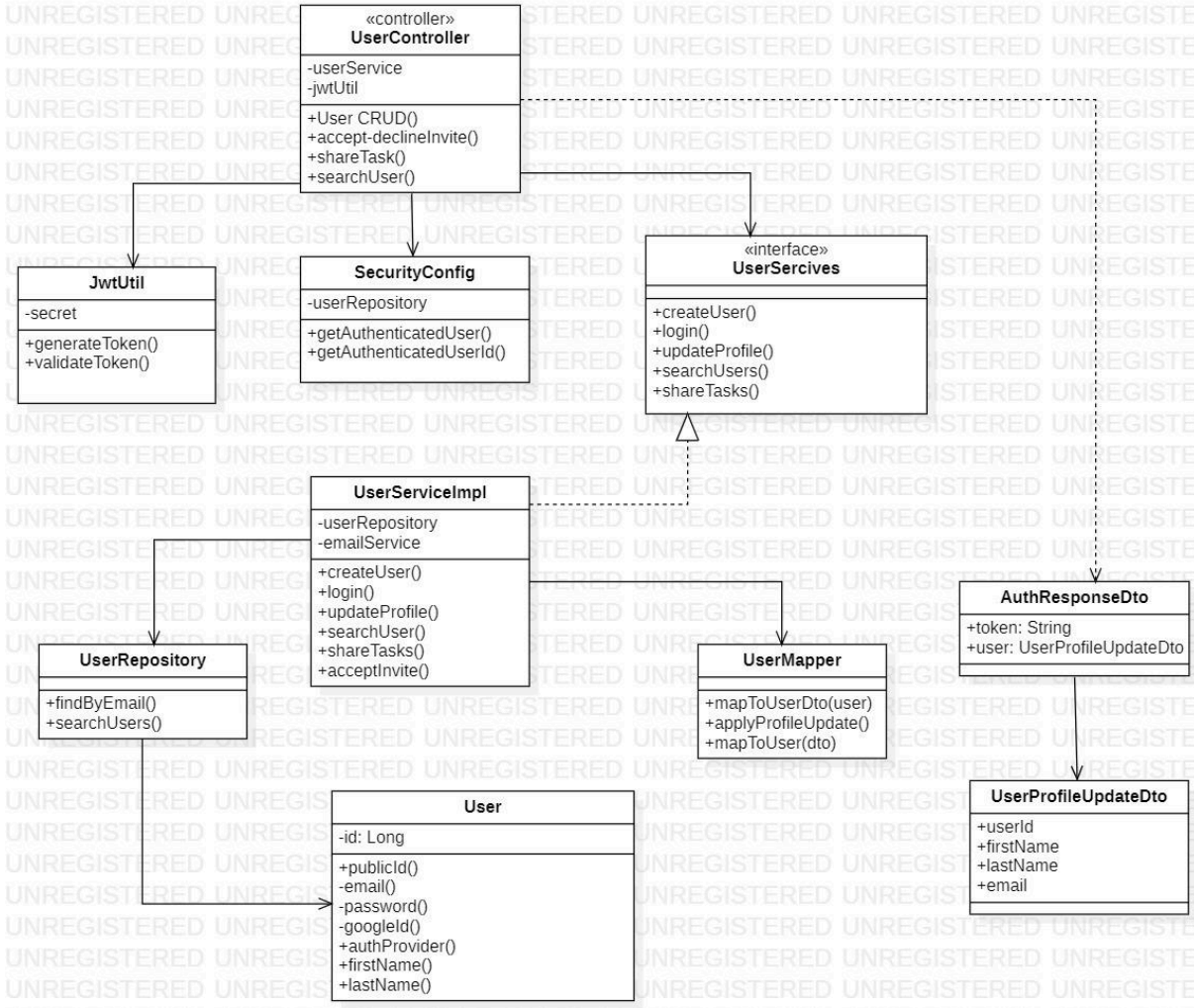
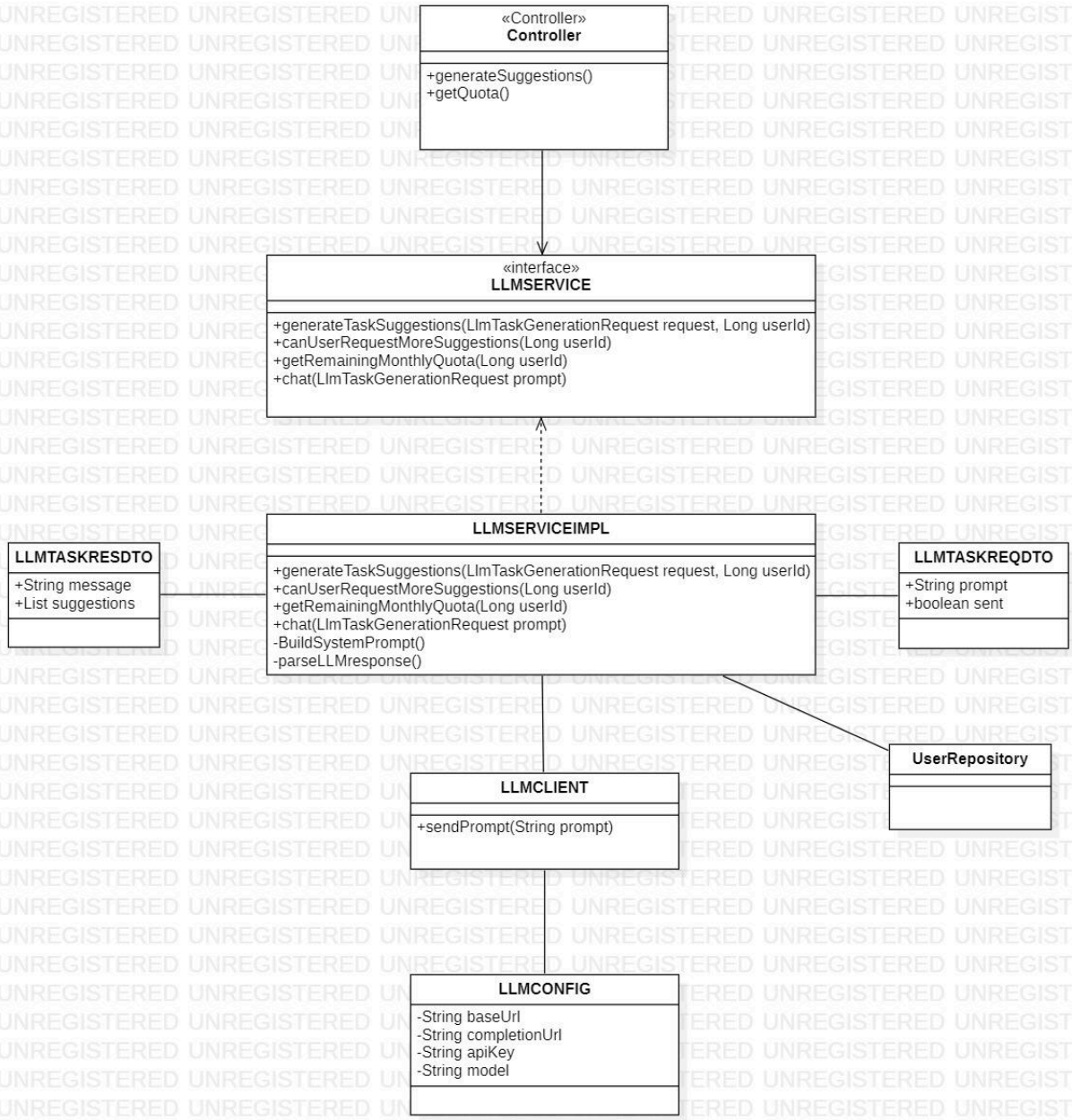
Assignment 09



Explanations:

1. **`UserController`**: Acts as the system's entry point. It manages HTTP requests for user CRUD operations, task sharing, and searching.
2. **`SecurityConfig`**: Manages the security context, providing methods to retrieve the currently authenticated user's details from the `UserRepository`.
3. **`UserServices` (Interface)**: Defines the contract for user-related actions, including `login()`, `createUser()`, and `shareTasks()`.
4. **`UserServiceImpl`**: The concrete implementation of the service interface. It orchestrates data flow by communicating with the `UserRepository` and uses an `emailService` for notifications.
5. **`UserRepository`**: The interface for database interactions, featuring custom query methods like `findByEmail()` and `searchUsers()`.
6. **`UserMapper`**: A utility used to transform data between the internal **User Entity** and external **Data Transfer Objects (DTOs)** to ensure data encapsulation.



Explanations:

1. **`Controller`**: Serves as the API entry point, exposing endpoints like `generateSuggestions()` and `getQuota()`.
2. **`LLMSERVICE` (Interface) & `LLMSERVICEIMPL`**: The core logic layer for servicing users for the LLM.

3. The implementation includes private helper methods like `BuildSystemPrompt()` and `parseLLMResponse()` to handle the specifics of AI interaction.
4. **`LLMCLIENT`**: A dedicated client responsible for the low-level execution of sending prompts to the external AI provider via `sendPrompt()`.
5. **`LLMCONFIG`**: Stores the necessary environment variables for the connection, including the `baseUrl`, `apiKey`, and the specific `model` version being used.
6. **`UserRepository`**: Integrated directly into the service implementation to track and verify user-specific data, likely for quota management.
7. **DTOs (`LLMTASKREQDTO` & `LLMTASKRESDTO`)**: Standardized objects for data transfer. The **Request DTO** carries the user prompt, while the **Response DTO** returns the generated message and a list of suggestions